

Hld Questions

Generated on February 05, 2026 at 01:29 PM | 58 entries

Q1. Design a URL Shortener (TinyURL)

[EASY] | Category: System Design

Answer: Designing a URL Shortener requires handling high read traffic and ensuring low latency.

Requirements:

- Functional:** shorten(longUrl) -> shortUrl, redirect(shortUrl) -> longUrl.
- Non-Functional:** Highly available, low latency, unique keys.

Capacity Estimation: If we have 100M new URLs/month and 100:1 read/write ratio. Storage needs for 5 years: $100M * 12 * 5 * 500\text{bytes} = 3\text{TB}$. This fits easily in a modern NoSQL cluster.

High Level Architecture:

- Clients:** Web/Mobile.
- Load Balancer:** Distributes requests.
- App Server:** Runs the shortening logic.
- Database:** A NoSQL store (like Cassandra or DynamoDB) is preferred for speed and scalability. We store `<ShortKey, LongURL>`.
- Cache:** Redis to store hot URLs (20% of URLs generate 80% of traffic).

Key Generation: We cannot just use random strings (collisions). We use a **Key Generation Service (KGS)**. This service pre-generates unique 6-character Base62 strings and stores them in a 'Unused Key' table. App servers fetch a batch of keys to memory, ensuring instant assignment without DB locking.

Q2. Design YouTube (Video Sharing Platform)

[HARD] | Category: System Design

Answer: Designing YouTube involves handling massive storage (blobs) and streaming throughput.

Core Components:

- Upload Service:** Handles incoming video files. Pushes to a processing queue.
- Transcoder (Worker):** Converts raw video into multiple formats (MP4, WebM) and resolutions (360p, 720p, 1080p).
- Blob Storage:** S3 or GCS for storing the actual video files.
- Metadata DB:** SQL/NoSQL to store Video Title, Description, UserID.
- CDN (Content Delivery Network):** The most critical part. Popular videos are pushed to edge servers globally to reduce latency.

Streaming Protocol: We use **Adaptive Bitrate Streaming (HLS/DASH)**. The client detects bandwidth. If internet is slow, it requests 360p chunks. If fast, it switches to 1080p chunks instantly.

Deduplication: Before uploading, we calculate the hash of the file. If it exists, we don't upload again (prevents piracy and saves storage).

Q3. Design a Chat System (WhatsApp)

[MEDIUM] | Category: System Design

Answer: A Chat System requires real-time delivery and extreme consistency. **Architecture:** 1. **Connection:** We use **WebSockets** for persistent, bi-directional connections. HTTP is too slow (polling). 2. **Chat Server:** Manages active connections. When User A sends a message, the server finds the WebSocket connection for User B and pushes the message. 3. **Presence Service:** Tracks if a user is Online/Offline/Last Seen. 4. **Store:** We need a write-heavy database. **HBase** or **Cassandra** is standard (WhatsApp uses a customized Erlang DB, Discord uses ScyllaDB). 5. **Push Notifications:** If User B is offline, the message goes to the Notification Service (FCM/APNS). **Security:** **End-to-End Encryption**. The server only relays encrypted binary data. It cannot read the messages. Keys are stored only on user devices.

Q4. Design Instagram (Photo Feed)

[MEDIUM] | Category: System Design

Answer: Instagram is a Read-Heavy system (Read:Write ratio ~ 100:1). The core challenge is **News Feed Generation**. **Core Components:** 1. **Image Storage:** S3/GCS. 2. **Metadata:** Postgres (Users, Photo info). 3. **Feed Service:** Generates the timeline. 4. **Graph DB (Neo4j):** To manage 'Follow' relationships efficiently. **Feed Generation Strategy:** 1. **Pull Model (Fan-out on Load):** When user refreshes, query all followings, get latest photos, merge, sort. **Con:** Slow for the user. 2. **Push Model (Fan-out on Write):** When User A posts, the system pushes the PhotoID to the 'Feed List' of all followers immediately in a Redis cache. When followers open the app, the feed is pre-built. **Con:** Slow for celebrities with 10M followers. 3. **Hybrid:** Push for normal users, Pull for celebrities.

Q5. Design a Rate Limiter

[HARD] | Category: System Design

Answer: A Rate Limiter prevents abuse and ensures fair usage. It is typically a middleware or part of the API Gateway (like Kong or Zuul). **Requirements:** 1. Allow 100 requests/minute per API Key. 2. Return HTTP 429 (Too Many Requests) if exceeded. 3. Low latency (< 5ms). **Architecture:** 1. **Client:** Sends request. 2. **Load Balancer:** Routes to API Gateway. 3. **Rate Limiter:** Checks Redis. 4. **Service:** Processes request if allowed. **Storage:** We use **Redis** because it is in-memory and supports atomic operations (INCR, EXPIRE). Database is too slow. **Algorithms:** 1. **Fixed Window:** Resets every minute. Susceptible to spikes at the edges. 2. **Sliding Window Log:** Stores timestamps. Accurate but memory expensive. 3. **Token Bucket:** Best balance. Tokens are added at rate r . Request consumes 1 token.

Q6. Design a Web Crawler

[HARD] | *Category: Distributed Systems*

Answer: A Web Crawler (Google Bot) indexes the internet. **Components:** 1. **Seed URLs:** Starting point (e.g., cnn.com, wikipedia.org). 2. **URL Frontier:** A massive priority queue managing which URL to visit next. 3. **DNS Resolver:** Caches IP addresses to avoid DNS lookup latency. 4. **Fetcher:** Downloads HTML. 5. **Parser:** Extracts links. 6. **Dedup:** Checks if content/URL is seen before. 7. **Storage:** BigTable (Google) or HBase. **Politeness:** We must not DDoS a server. The Frontier ensures we don't hit the same domain multiple times per second. We use a queue-per-domain mapping. **Duplicate Detection:** We use **Bloom Filters** or **Checksums (MD5/SHA)**. Storing 1 Billion URLs in memory is impossible. A Bloom Filter tells us "Definitely No" or "Maybe Yes" with very little memory.

Q7. Design Uber (Ride Sharing)

[HARD] | *Category: System Design*

Answer: Uber matches Riders with Drivers based on Geo-Location. ****Core Challenges:**** 1. Real-time location updates (Drivers move every second). 2. Efficient Spatial Search (Find drivers within 2km). 3. Consistency (Ride matching). ****Architecture:**** 1. ****Driver App:**** Sends GPS every 4 seconds. 2. ****Location Service:**** Updates location in memory (Redis). 3. ****Map Service:**** Handles routing and ETA. 4. ****Match Service:**** The brain. Pairs rider/driver. ****Spatial Indexing:**** We cannot use SQL `WHERE lat > x AND long < y`. It's too slow. We use ****QuadTrees**** (or Google S2). A QuadTree divides the map into 4 quadrants recursively. This allows `O(log N)` search for "nearby points".

Q8. Design Google Drive (File Storage)

[MEDIUM] | *Category: Distributed Systems*

Answer: Google Drive is a read-heavy, sync-heavy system focusing on data integrity and consistency. ****Core Components:**** 1. ****Block Server:**** Splits files into chunks (e.g., 4MB). 2. ****Cloud Storage:**** S3/GCS stores the chunks. 3. ****Metadata DB:**** SQL DB stores file hierarchy and chunk mappings. 4. ****Synchronization Service:**** Handles conflicting edits. ****Upload Flow:**** We don't upload the whole file. We split it. 1. Calculate hash of each chunk. 2. Check if chunk exists (Deduplication). 3. Upload only new chunks. ****Consistency:**** We use ****Differential Synchronization**** or simple versioning. If Client A and B edit offline and come online, the server detects a version conflict and asks the user to resolve, or creates "Conflicted Copy".

Q9. Design a Typeahead (Autocomplete)

[MEDIUM] | *Category: System Design*

Answer: Typeahead provides search suggestions (like Google Search). ****Requirements:**** 1. Low latency (< 100ms). 2. Prefix matching. 3. Ranking (show most popular searches first). ****Data Structure:**** We use a ****Trie (Prefix Tree)****. Root -> 'a' -> 'p' -> 'p' -> 'l' -> 'e'. Each node stores the top 5 popular queries ending at that node. ****Optimization:**** We cannot traverse the whole tree in real-time. We ****cache the top 5 results at each node****. When user types "app", we go to node 'p', and instantly return the pre-calculated list ['apple', 'app store', 'apply'].

Q10. Design a Key-Value Store (Dynamo/Cassandra)

[HARD] | *Category: Distributed Systems*

Answer: We need to design a distributed KV store with high availability. **Core Concepts:** 1. **Partitioning:** We cannot fit all data on one server. We use **Consistent Hashing** to distribute keys across a ring of servers. 2. **Replication:** To handle failure, every key is replicated to N servers (usually 3). 3. **Consistency Model:** We use **Quorum Consensus**. * $N = 3$ (Replicas) * $W = 2$ (Write Quorum - wait for 2 acks) * $R = 2$ (Read Quorum - wait for 2 responses) * If $W + R > N$, we have strong consistency. **Conflict Resolution:** If two writes happen simultaneously on different replicas, we use **Vector Clocks** or Last-Write-Wins (LWW) to resolve.

Q11. Design a Notification Service

[MEDIUM] | *Category: Distributed Systems*

Answer: A Notification Service acts as a centralized platform to send emails, SMS, and Push Notifications. It decouples the 'trigger' from the 'delivery'. **Core Architecture:** 1. **Notification Gateway:** The entry point (API). 2. **Validator & Prioritizer:** Checks payload size and assigns priority (OTP = High, Marketing = Low). 3. **Message Queue (RabbitMQ/Kafka):** Buffers requests to handle bursts. 4. **Workers:** Consumers that pull messages and call external providers (Twilio, SendGrid, FCM). 5. **Notification Log:** Database (Cassandra) to track status (Sent/Failed). **Reliability:** We use a **Retry Mechanism** with exponential backoff. If Twilio is down, we don't drop the message; we requeue it with a delay. **Deduplication:** To prevent spamming a user, we check a Redis cache. If a 'Welcome' email was sent to UserID 123 in the last 5 minutes, we suppress duplicates.

Q12. Design a Distributed Job Scheduler

[HARD] | *Category: Distributed Systems*

Answer: A Distributed Job Scheduler (like Cron for a cluster) executes tasks at specific times or intervals across many machines. **Requirements:** 1. **High Availability:** If the scheduler crashes, jobs must still run. 2. **Exactly-Once Execution:** A job shouldn't run twice (double billing). 3. **Scalability:** Handle millions of jobs. **Architecture:** 1. **Job Submission API:** Accepts tasks (One-time or Recurring). 2. **Job Store (DB):** Stores task metadata (Postgres/MySQL). 3. **Scheduler Nodes:** Master node checks DB for 'due' jobs. 4. **Worker Nodes:** Execute the actual code. 5. **Coordination:** Zookeeper or Redis for Leader Election. **The Polling Problem:** Instead of polling the DB every second (inefficient), we use a **Hierarchical Timing Wheel** (efficient memory structure) or push 'near-future' jobs into a Redis Sorted Set.

Q13. Design a Metrics Monitoring System (Datadog)

[HARD] | Category: System Design

Answer: A Monitoring System collects, aggregates, and visualizes time-series data. ****Core Components:****

1. ****Agent:**** Runs on every host. Collects CPU/RAM/Logs. Pushes to server.
2. ****Ingestion Service:**** High-throughput API (Kafka) to accept data.
3. ****Time Series Database (TSDB):**** InfluxDB or Prometheus. Optimized for writing millions of points/sec.
4. ****Query Engine:**** Visualizes data (Grafana).
5. ****Alerting Service:**** Checks rules ("CPU > 90%") and notifies.

****Data Model:**** Data is stored as `(MetricName, Timestamp, Value, Tags)`. Example: `(cpu\_usage, 1625000000, 85%, {host: i-123, region: us-east})`.

****Aggregation:**** We cannot store raw data forever. We use ****Rollups (Downsampling)****. * Raw data kept for 7 days. * 1-minute averages kept for 30 days. * 1-hour averages kept for 1 year.

Q14. Design Google Maps (Navigation)

[HARD] | Category: System Design

Answer: Google Maps is a massive geospatial system handling Routing, Navigation, and Map Rendering. ****Core Components:****

1. ****Map Data Service:**** Stores map tiles (images) and vector data (road network).
2. ****Navigation Service:**** Calculates shortest path (A* or Dijkstra).
3. ****Location Service:**** Tracks user location.
4. ****Traffic Service:**** Analyzes real-time speed to update ETAs.

****Data Storage:**** * ****Graph:**** Roads are nodes/edges in a Graph DB. * ****Tiles:**** Map images are stored in a Pyramid structure (Zoom levels) in Object Storage (S3) and cached heavily in CDNs. ****Routing Algorithm:**** We don't run Dijkstra on the whole world. We use ****Partitioning**** (Hierarchical Pathfinding). We calculate routes between "Key Cities" first, then refine local streets.

Q15. Design Tinder (Matching System)

[MEDIUM] | Category: System Design

Answer: Tinder relies on ****Geospatial Search**** and ****Active Recommendation****. ****Core Components:****

1. ****User Profile DB:**** Photos, Bio.
2. ****Geo-Location Service:**** Tracks user movement (QuadTree/Geohash).
3. ****Recommendation Engine:**** The brain. Selects potential matches.
4. ****Match Service:**** Handles 'Swipes'. Checks for mutual likes.

****Storage:**** * ****Images:**** S3 + CDN. * ****Swipes:**** NoSQL (Cassandra/DynamoDB) is critical. We store billions of swipes. * Table: `Likes` (UserA, UserB, Timestamp). ****Algorithm:**** When User A opens app: 1. Find users within 10km (Geo Service). 2. Filter out users A has already swiped (Bloom Filter). 3. Rank remaining users (ML Model: Attractiveness score, Activity score).

Q16. Design Amazon (E-Commerce)

[HARD] | *Category: System Design*

Answer: An E-commerce system involves **Inventory Management**, **Cart**, **Order Processing**, and **Checkout**. **Core Components:** 1. **Catalog Service:** Read-heavy. Product details. (Elasticsearch + Cache). 2. **Cart Service:** Write-heavy. Temporary state. (Redis/DynamoDB). 3. **Order Service:** High consistency. Orchestrates the purchase. 4. **Inventory Service:** Manages stock counts. **The Inventory Challenge:** How to prevent overselling? * **Optimistic Locking:** When decreasing stock, `UPDATE stock SET count = count - 1 WHERE id = 1 AND count > 0`. If row count is 0, transaction fails. * **Reservation:** When user proceeds to Checkout, we "reserve" items for 10 mins. If payment fails, we release them.

Q17. Design a Real-time Gaming Leaderboard

[MEDIUM] | *Category: Distributed Systems*

Answer: A Leaderboard needs to display top players and surrounding rank in real-time. **Requirements:** 1. **Low Latency:** Updates reflect instantly. 2. **Scale:** Millions of players. 3. **Operations:** `UpdateScore`, `GetTop10`, `GetMyRank`. **Architecture:** Standard SQL (`ORDER BY score`) is too slow ($O(N \log N)$). We use **Redis Sorted Sets (ZSET)**. * ZSET uses a Skip List internally. * `ZADD leaderboard <score> <user>`: $O(\log N)$. * `ZREVRANGE leaderboard 0 9`: $O(\log N)$. Gets Top 10. * `ZRANK leaderboard <user>`: $O(\log N)$. Gets rank. **Scaling Redis:** Redis is single-threaded. For 100M users, we **Sharding**. * Shard by score range? (Bad, hotspots). * Shard by UserID? (Good for updates, bad for querying Top K). * **Solution:** The **Scatter-Gather** approach. Shard users across N Redis instances. To get Top 10, query ALL instances for their Top 10, merge results in app server, and return global Top 10.

Q18. Design Collaborative Editing (Google Docs)

[HARD] | *Category: Distributed Systems*

Answer: This requires multiple users editing the same document simultaneously without conflicts. **Core Challenges:** 1. **Concurrency:** User A types "Hello", User B types "World" at the same time. 2. **Latency:** User needs to see their own typing instantly (Optimistic UI). **Algorithm:** We use **Operational Transformation (OT)** or **CRDT (Conflict-free Replicated Data Types)**. * **OT:** Used by Google Docs. It transforms operations based on context. If A inserts at index 0, B's insert at index 5 must be shifted to index 6. * **CRDT:** Newer, decentralized approach. Better for P2P. **Architecture:** 1. **WebSocket:** For real-time character streaming. 2. **Session Server:** Maintains the in-memory state of the document. 3. **Operation Log:** Stores every single keystroke/change. 4. **Snapshotting:** Periodically saves the full document to S3/DB to speed up loading.

Q19. Design a Distributed Lock Manager (Chubby)

[HARD] | Category: Distributed Systems

Answer: In a distributed system, we need to ensure only ONE process accesses a resource at a time (e.g., writing to a file, processing a payment). **Requirements:** 1. **Mutual Exclusion:** Only one client holds the lock. 2. **Deadlock Free:** If client crashes, lock must release. 3. **Fault Tolerance:** Lock service itself must not fail. **Tools:** We don't build this from scratch usually. We use **Zookeeper** (Zab protocol), **Etcd** (Raft), or **Redis** (Redlock). **Mechanism (Zookeeper):** 1. Clients try to create an **Ephemeral Sequential Node** under `/lock`. 2. The client with the lowest sequence number gets the lock. 3. Others "Watch" the node previous to them. 4. If the lock holder crashes, the ephemeral node disappears. The next watcher is notified.

Q20. Design Netflix (Video Streaming)

[MEDIUM] | Category: System Design

Answer: Netflix is similar to YouTube but focuses on **Long-form content**, **DRM**, and **Personalization**. **Core Components:** 1. **CDN (Open Connect):** Netflix builds its own custom CDN hardware installed inside ISP datacenters. This is key. 2. **Control Plane:** AWS. Handles login, billing, browsing. 3. **Data Plane:** CDNs. Handles video streaming. 4. **Transcoding:** Supports thousands of device types. **High Level Flow:** Client logs in (AWS). Browses catalog. Clicks Play. AWS determines the best Open Connect Appliance (OCA) near the user (inside their ISP). Client streams from that OCA. **Resilience:** Netflix uses **Chaos Monkey** to randomly kill services in production to ensure fallback mechanisms work.

Q21. Design a News Feed (Facebook)

[MEDIUM] | Category: System Design

Answer: Designing a News Feed involves aggregating posts from friends and ranking them. **Models:** 1. **Pull (Fan-out on Load):** Good for low traffic. Query DB for all friends' posts. 2. **Push (Fan-out on Write):** Good for read-heavy. When A posts, write to all friends' timelines. 3. **Hybrid:** The industry standard. Push for normal users. Pull for celebrities. **Ranking:** We don't just show chronological. We score posts (EdgeRank). $\text{Score} = \text{Affinity} * \text{Weight} * \text{TimeDecay}$. * **Affinity:** How much you interact with the author. * **Weight:** Video > Image > Text. * **Time:** Newer is better.

Q22. Design Zoom (Video Conferencing)

[HARD] | *Category: System Design*

Answer: Zoom requires **Low Latency** (Real-time). We cannot use TCP (retransmission delays). We use **UDP**. **Protocols:** * **WebRTC:** Standard for browser P2P. * **RTP/RTCP:** Real-time Transport Protocol. **Architecture:** 1. **P2P (Mesh):** User A sends to B, C, D. Bandwidth = (N-1) streams. Bad for large groups. 2. **SFU (Selective Forwarding Unit):** Everyone sends stream to Server. Server forwards to others. Best for Zoom. 3. **MCU (Multipoint Control Unit):** Server mixes video into one stream. Heavy CPU. **Zoom uses SFU.** The client uploads 1 stream. The server routes it to 100 participants.

Q23. Design an Ad Click Aggregator

[HARD] | *Category: Distributed Systems*

Answer: We need to track billions of ad clicks for billing. This requires **Exactly-Once Processing** and **High Throughput**. **Architecture:** 1. **Ad Server:** Logs click to Kafka. 2. **Stream Processor (Flink/Spark):** Consumes Kafka. Aggregates clicks per AdID per minute. 3. **OLAP Database (ClickHouse/Druid):** Stores aggregated data for reporting. **Challenge:** * **Duplicate Clicks:** User clicks twice. Bot clicks. * **Late Data:** Click happens at 9:59, arrives at server at 10:01. **Solution:** We use **Windowing** (Tumbling Windows) in the stream processor. We handle late events using Watermarks.

Q24. Design a Distributed Cache (Redis)

[HARD] | *Category: System Design*

Answer: We need a key-value store faster than a DB. **Core Concepts:** 1. **Eviction Policy:** LRU (Least Recently Used). 2. **Partitioning:** Consistent Hashing. 3. **High Availability:** Master-Slave Replication with Sentinel. **The Thundering Herd Problem:** If a popular cache key expires, 1000 requests hit the DB simultaneously. **Solution:** **Request Coalescing.** The cache client waits if a fetch is already in progress. Or **Probabilistic Early Expiration** (refresh before it actually expires).

Q25. Design a Ticket Booking System (Ticketmaster)

[MEDIUM] | Category: System Design

Answer: The critical part is **Concurrency**. 100,000 people trying to buy 100 tickets. **Architecture:** 1. **Waiting Room (Queue):** Throttles users. 2. **Booking Service:** Handles logic. 3. **Active Reservation Service (Redis):** Holds seat for 5 mins. **Concurrency Strategy:** We cannot rely on DB transactions alone (too slow). We use **Redis Lua Scripts** or **Distributed Locks** on the specific seat. `SET seat_123 booked NX EX 300`` (Set if Not Exists, Expire 300s). **Database:** Eventually consistent for browsing (Read Replica). Strongly consistent for Booking (Master).

Q26. Design a Nearby Friends Service

[MEDIUM] | Category: Distributed Systems

Answer: We need to show friends within 5 miles. Location updates are frequent. **Architecture:** 1. **Location Ingestion:** WebSocket connection. 2. **Redis Pub/Sub:** To notify friends? No, too expensive (N^2). 3. **Geohash:** Indexing. **Optimization:** We don't need perfect accuracy. **Approach:** User A is in Geohash `u4pru``. Friends Query: "Who is in `u4pru`` or neighbors?" * We store `Key: UserID, Value: Geohash` AND `Key: Geohash, Value: List<UserID>``. **Ephemeral Storage:** Location history is expensive. We only store the *current* location in Redis. History goes to cold storage (Cassandra) async.

Q27. Design a Distributed Search Engine (Elasticsearch)

[HARD] | Category: System Design

Answer: We need to search documents by keyword with full-text features (fuzzy, ranking). **Core Components:** 1. **Inverted Index:** Maps `Word -> List<DocumentIDs>``. * `"Apple" -> [D1, D3]` * `"Banana" -> [D2]` 2. **Sharding:** Split index across nodes. 3. **Coordinator Node:** Receives query, fans out to shards, merges results. **Write Flow:** Data is immutable (Lucene segments). 1. Write to buffer. 2. Flush to segment file on disk. 3. Segment is now searchable. **Ranking:** TF-IDF (Term Frequency - Inverse Document Frequency). If a user searches "The Apple", "Apple" is rare (high weight), "The" is common (low weight).

Q28. Design a Code Deployment System

[MEDIUM] | Category: System Design

Answer: A CI/CD system to deploy code to 10,000 servers. **Requirements:** 1. **Rollback:** If bug found, revert instantly. 2. **Zero Downtime:** Users shouldn't see 503 errors. 3. **Speed:** Deploy in minutes. **Strategies:** 1. **Rolling Deployment:** Update 10% servers. If healthy, next 10%. 2. **Blue-Green:** Create full new cluster (Green). Switch Load Balancer. 3. **Canary:** Deploy to 1% users. Monitor metrics. **Architecture:** 1. **Build Server:** Creates Artifact (Docker Image). 2. **Repo:** Artifactory/ECR. 3. **Deploy Agent:** Runs on every server. Pulls artifact. 4. **Orchestrator:** Manages the rollout.

Q29. Design a Fantasy Sports System

[MEDIUM] | *Category: System Design*

Answer: Users draft players. Real-world performance updates scores. ****Challenges:**** 1. ****Real-time Scoring:**** 10M users affected by 1 Touchdown. 2. ****Consistency:**** Scores must be accurate. ****Architecture:**** 1. ****Data Provider:**** External API (Stats Inc). 2. ****Ingestion:**** Kafka. 3. ****Calculation Engine:**** Updates Player stats. 4. ****Fan-out:**** Updates User Teams. ****Optimization:**** We cannot query "Select all teams with Player X" (too slow). We pre-calculate. * Store `Player -> List<TeamIDs>`. * When Player X scores, fetch TeamIDs. Update Team Scores in Redis.

Q30. Design a Fraud Detection System

[HARD] | *Category: System Design*

Answer: We need to detect suspicious payments in < 200ms. ****Architecture:**** 1. ****Rule Engine:**** Checks hard rules (e.g., "Transaction > \$10k"). 2. ****ML Model:**** Checks probability (e.g., "User usually buys coffee, not diamonds"). 3. ****Graph DB:**** Checks network (e.g., "Is this IP linked to known fraudsters?"). ****Pipeline:**** Payment Gateway -> Kafka -> Fraud Service -> (Parallel: Rules, ML, Graph) -> Decision. ****Speed:**** ML Models (Random Forest) must run fast. Features (e.g., "Count of transactions in last hour") are pre-calculated in Redis.

Q31. Design an Object Storage Service (Amazon S3)

[HARD] | Category: Distributed Systems

Answer: Designing an Object Storage service like S3 is fundamentally different from designing a file system. We are not managing a hierarchy of folders but a flat namespace of **Buckets** and **Objects**.
The Elevator Pitch: The system must handle **immutable binary data** (blobs) ranging from a few bytes to terabytes. The core architecture relies on decoupling the **Metadata** (who owns this, what is it named?) from the **Block Storage** (the actual bytes).
Key Architectural Components: 1. **Front-end Load Balancers:** Handle API requests (REST). 2. **Metadata Service:** A highly consistent key-value store (like CockroachDB or sharded MySQL) that maps `BucketName/ObjectName` to `DataNodeIDs`. 3. **Data Nodes:** Commodity servers with massive hard drives. They run a daemon to manage local disk I/O. 4. **Placement Driver:** Decides where to replicate data to ensure durability (e.g., Erasure Coding).
Durability Strategy: We do not use RAID. We use **Erasure Coding (EC)**. Instead of storing 3 full copies (300% storage overhead), we split the file into `n` data chunks and `k` parity chunks. This allows us to recover data even if multiple disks fail, with only ~1.5x storage overhead.

Q32. Design a Content Delivery Network (CDN)

[HARD] | Category: System Design

Answer: A CDN is a network of geographically distributed proxy servers. The goal is to serve content from a server closest to the user (The Edge) to minimize latency.
The Elevator Pitch: The core of a CDN is **DNS Geo-Routing**. When a user looks up `cdn.netflix.com`, the DNS server does not return a fixed IP. It analyzes the user's ISP and location, then returns the IP of the nearest **Point of Presence (PoP)**.
Key Components: 1. **Edge Servers:** SSD-heavy servers located in ISPs. They cache content (LRU). 2. **Origin Server:** The 'Source of Truth' (e.g., S3 bucket). 3. **Control Plane:** Manages configuration, invalidation, and health checks. 4. **Routing System:** The DNS magic that maps User -> Best Edge.
Caching Strategy: We use a **Pull Model**. The Edge server is empty initially. When the first user requests `video.mp4`, the Edge fetches it from the Origin, saves it, and serves it. Subsequent users get the cached copy.

Q33. Design a Webhook Delivery System (Stripe)

[MEDIUM] | Category: Distributed Systems

Answer: Webhooks are "User-Defined HTTP Callbacks". When an event happens in our system (e.g., Payment Succeeded), we make an HTTP POST request to a URL configured by the user.
The Elevator Pitch: This is effectively a **Reverse API**. Instead of the user calling us, we call them. The challenge is reliability. User servers are often flaky, slow, or down. We need a robust **Queue-based Worker System** with **Exponential Backoff**.
Architecture: 1. **Event Generator:** Emits events (Payment Service). 2. **Webhook Service:** Persists the event to DB. 3. **Dispatcher:** Pushes jobs to Redis/Kafka. 4. **Delivery Workers:** Pull jobs and execute HTTP POST. 5. **Scheduler:** Retries failed jobs. **Security:** We must prove to the user that the request actually came from us. We sign the payload using an **HMAC** (Hash-based Message Authentication Code) shared secret.

Q34. Design a Distributed Tracing System (Jaeger/Zipkin)

[MEDIUM] | Category: Distributed Systems

Answer: In Microservices, a single user request might traverse 20 different services. If the request is slow, how do we know *which* service is the bottleneck? **The Elevator Pitch:** A Distributed Tracing system assigns a unique **TraceID** to every request at the ingress (Load Balancer). This ID is propagated in HTTP Headers (`X-Trace-ID`) across all internal calls. Each service reports its start/end time (Spans) to a central collector. **Architecture:** 1. **Instrumentation SDK:** Library inside every microservice code (intercepts HTTP calls). 2. **Agent:** Runs on the host, buffers spans (UDP). 3. **Collector:** Validates and indexes spans. 4. **Storage:** Cassandra or Elasticsearch (Write-heavy). 5. **UI:** Visualizes the Timeline (Gantt Chart). **Sampling:** We cannot trace 100% of traffic (too much storage). We use **Probabilistic Sampling** (e.g., trace 0.1% of requests) or **Head-based Sampling**.

Q35. Design a Flash Sale System (High Concurrency)

[HARD] | Category: System Design

Answer: A Flash Sale (e.g., Black Friday, Ticket Sales) involves selling a limited inventory (100 iPhones) to millions of users in seconds. **The Elevator Pitch:** The challenge is **Database Contention**. Standard SQL locking (`UPDATE stock ...`) handles ~500 writes/sec. We need 100,000 requests/sec. We must move the "locking" logic **upstream** from the DB to Memory (Redis). **Architecture:** 1. **CDN/Static Page:** The landing page is static HTML cached on CDN. 2. **Queue/Waiting Room:** Throttles incoming traffic. 3. **Redis Limiter:** Decrements counters atomically. 4. **Async Order Processor:** Only successful Redis decrements are processed into Orders. **Core Rule:** **Never hit the database to check stock.** The database is for *recording* the sale, not checking availability.

Q36. Design a Code Collaboration Platform (GitHub)

[HARD] | Category: System Design

Answer: GitHub is a mix of a **Git Host**, **Social Network**, and **Project Management** tool. **The Elevator Pitch:** The core problem is storing Git repositories. A Git repo is a Directed Acyclic Graph (DAG) of objects (commits, trees, blobs). It's a file-system heavy workload. We cannot store Git objects in a SQL database effectively. We need a specialized **Distributed File System**. **Architecture:** 1. **Proxy Layer:** Handles SSH/HTTPS git commands. 2. **Git Workers:** Execute the actual `git pack-objects`, `git receive-pack`. 3. **Storage Layer:** Custom RPC (Remote Procedure Call) to disk. GitHub uses **Spokes** (a replication system). 4. **Metadata DB:** MySQL. Stores Pull Requests, Issues, Comments (Relational data). **Replication:** Standard disk replication (RAID) isn't enough. We replicate at the **Git Object level**.

Q37. Design a Multiplayer Game Server (Fortnite)

[HARD] | *Category: Game Design*

Answer: Real-time multiplayer games require **state synchronization** between 100 players at 60Hz (60 updates/sec). **The Elevator Pitch:** We cannot use standard HTTP. We use **UDP (User Datagram Protocol)** for speed. The server runs the **Game Loop**: Input -> Simulate -> Broadcast. The server is the "Authoritative Source of Truth" to prevent cheating. **Architecture:** 1. **Matchmaker:** Groups 100 players based on skill/latency. 2. **Dedicated Game Server (DGS):** A stateful process hosting ONE specific match. 3. **Gateway:** Routes packets. **Lag Compensation:** Network travel time (Ping) is unavoidable. If I shoot where you *were* 100ms ago, I miss. The server uses **Client-Side Prediction** and **Server Rewind**.

Q38. Design a Stock Exchange Matching Engine

[HARD] | *Category: System Design*

Answer: A Matching Engine matches Buy and Sell orders for stocks. It requires **Microsecond Latency** and **100% Determinism**. **The Elevator Pitch:** We cannot use a database transaction for every match (too slow). The entire Order Book (list of Buys/Sells) is kept in **In-Memory** data structures. A single thread processes the order book for one stock symbol to avoid locking overhead. **Architecture:** 1. **Gateway:** FIX Protocol adapter (Financial Standard). 2. **Sequencer:** Assigns a global sequence number to every order. 3. **Matching Engine:** The core logic. 4. **Market Data Publisher:** Broadcasts prices to the world. **Reliability:** If RAM is lost, the exchange dies. We use **Event Sourcing**. Every incoming order is written to a specialized Log (Journal) on NVMe SSDs before processing. On restart, we replay the log.

Q39. Design a Recommendation System (TikTok/Netflix)

[HARD] | *Category: System Design*

Answer: Recommendation Systems predict: "User U will like Item I". **The Elevator Pitch:** It's a funnel. We start with 1 Million items and narrow it down to 10. **Funnel Stages:** 1. **Candidate Generation (Retrieval):** Fast, crude selection. "Find all movies user's friends watched". (Reduces 1M -> 1000). 2. **Ranking (Scoring):** Heavy ML model. Predicts probability of click. (Reduces 1000 -> 10). 3. **Re-Ranking:** Business logic. "Don't show 10 horror movies in a row". **Architecture:** 1. **Offline Training:** BigQuery/Spark. 2. **Nearline:** Real-time user history (Kafka). 3. **Inference Service:** Serving the model.

Q40. Design a Metrics Alerting System (PagerDuty)

[MEDIUM] | Category: System Design

Answer: We need to ingest events from monitoring tools and wake up the on-call engineer via SMS/Phone. **The Elevator Pitch:** Reliability is the #1 requirement. The alerting system cannot go down when the main system goes down. It must be **Multi-Region**. It focuses on **On-Call Scheduling** and **Escalation Policies**. **Architecture:** 1. **Ingestion API:** Receives JSON alerts from Prometheus/Datadog. 2. **Deduplication Engine:** If 100 servers crash, don't send 100 SMS. Send 1 "Cluster Down". 3. **State Machine:** Manages Alert Lifecycle (Triggered -> Acknowledged -> Resolved). 4. **Notification Provider:** Twilio/SendGrid. **Escalation:** If Level 1 Engineer doesn't ACK in 5 mins, escalate to Level 2.

Q41. Design a Live Commenting System (Twitch/YouTube Live)

[MEDIUM] | Category: System Design

Answer: Live streams can generate 100k comments per second. We need to display them in real-time to millions of viewers. **The Elevator Pitch:** A standard database write (INSERT) for every comment will kill the DB. We use a **Write-Behind** pattern with massive buffering. For reading, we cannot broadcast every message to every user (N*M complexity). We use **Time-Bucketing** and **Sampling**. **Architecture:** 1. **Edge Gateway:** WebSocket termination. 2. **Message Broker:** Kafka (Partitioned by ChannelID). 3. **Stream Processor:** Aggregates/Filters spam. 4. **Fan-out Service:** Pushes to viewers. **The "Chatty" Problem:** If 1M viewers are watching, and 100k messages come in, we have 100 Billion push operations/sec. This is impossible. **Solution:** **Throttling/Sampling**. The UI can only render 50 msgs/sec. We drop the rest or show a random sample to the viewer.

Q42. Design a Dark Store Delivery System (Gorillas/Blinkit)

[HARD] | Category: System Design

Answer: Quick Commerce promises grocery delivery in 10-15 minutes. **The Elevator Pitch:** The constraint is Time. 15 minutes = 2 min packing + 10 min travel + 3 min buffer. The system relies on **Hyper-local Warehouses (Dark Stores)**. Each store covers a 2km radius. Inventory needs to be **Real-time**. If the app says "Milk available", it *must* be there. **Architecture:** 1. **User App:** Geo-fenced catalog (Only shows items in the nearby dark store). 2. **WMS (Warehouse Management System):** Tablet app for packers. 3. **Assignment Engine:** Matches Order -> Driver. 4. **Inventory Engine:** High consistency. **Inventory Strategy:** We use **Pessimistic Locking** or atomic decrements (Redis) on the specific store's stock.

Q43. Design a Digital Signature Service (DocuSign)

[MEDIUM] | Category: System Design

Answer: This system allows users to upload PDFs, drag-and-drop signature fields, and sign legally. **The Elevator Pitch:** Key requirements are **Security (Non-repudiation)** and **Audit Trails**. We are not just storing files; we are modifying them (flattening signatures into the PDF). We use **PKI (Public Key Infrastructure)** to cryptographically sign the document. **Architecture:** 1. **Document Service:** Handles Upload/Conversion (LibreOffice/Pandoc). 2. **Canvas UI:** Coordinate mapping (X, Y pixels). 3. **Signing Engine:** Applies X.509 Digital Certificates. 4. **Notification:** Emails next signer. **The Audit Trail:** We log IP, Timestamp, Email, and Browser Agent for every view/sign action. This log is appended to the final PDF as a "Certificate of Completion".

Q44. Design a VPN Service (NordVPN)

[HARD] | Category: System Design

Answer: A VPN tunnels user traffic through an encrypted server to hide IP and bypass geo-blocks. **The Elevator Pitch:** It's a proxy on steroids. The client creates a virtual network interface (TUN/TAP). All OS traffic is routed into this interface, encrypted, and sent to the VPN Server. The VPN Server decrypts it and forwards it to the Internet (NAT). **Architecture:** 1. **Client App:** Handle handshake and routing table. 2. **VPN Gateway:** The servers (Exit Nodes). 3. **Auth Service:** Radius/LDAP. 4. **Session Manager:** Tracks active connections for billing. **Protocols:** * **OpenVPN:** Robust, uses TCP/UDP + SSL. Slow. * **WireGuard:** Modern, kernel-level, extremely fast, stateless. **No-Logs Policy:** The architecture must ensure **RAM-only** operation for Gateways. No hard drive writing for traffic logs.

Q45. Design a Privacy Analytics System (GDPR Compliant)

[MEDIUM] | Category: System Design

Answer: We need to track website usage (Page Views) without storing PII (Personally Identifiable Information) like IPs or Cookies, to comply with GDPR/CCPA. **The Elevator Pitch:** We cannot use cookies. We use **Fingerprinting** or **Ephemeral Sessions**. The raw data is hashed immediately. We process data in **Aggregated Form** only. **Architecture:** 1. **Collector Script:** Tiny JS (1KB). 2. **Ingestion API:** Receives `(UserAgent, URL, Referrer)`. 3. **Anonymizer:** The core privacy component. 4. **TSDB:** ClickHouse (Columnar store). **Data Hashing:** Daily Salt. `Hash = SHA256(IP + UserAgent + DailySalt)`. Because the Salt changes every 24h, we cannot track a user across days. This enables "Daily Unique Visitors" but prevents long-term surveillance.

Q46. Design an IoT Ingestion Platform

[HARD] | *Category: System Design*

Answer: We need to collect telemetry (Temperature, Speed, Pressure) from 1 Million sensors every second. **The Elevator Pitch:** HTTP is too heavy (headers overhead). We use **MQTT** (Message Queuing Telemetry Transport), a lightweight binary protocol over TCP. The backend requires a massive write-heavy pipeline. **Architecture:** 1. **Device:** Runs MQTT Client. 2. **IoT Gateway:** MQTT Broker (VerneMQ / Mosquitto). Maintains connections. 3. **Kafka:** Buffers data. 4. **Rule Engine:** Real-time alerts ("Temp > 100"). 5. **Cold Storage:** S3 (Parquet format) for history. 6. **Hot Storage:** InfluxDB for dashboards. **Connectivity:** Sensors have intermittent connectivity. MQTT supports **QoS** (Quality of Service) levels. QoS 1 ensures delivery at least once (store and forward).

Q47. Design a Container Orchestrator (Kubernetes-lite)

[HARD] | *Category: Distributed Systems*

Answer: We need a system to manage Docker containers across a cluster of servers: Scheduling, Healing, Scaling. **The Elevator Pitch:** We need a **Control Plane** (Brain) and **Worker Nodes** (Muscle). The state of the cluster is stored in **Etcd** (Distributed Key-Value Store). The paradigm is **Declarative**: User says "I want 3 copies of Nginx". The Controller Loop works to make Reality == Desired. **Architecture:** 1. **API Server:** The entry point. 2. **Scheduler:** Decides *where* to put a pod (Bin Packing). 3. **Controller Manager:** Detects crashes and restarts pods. 4. **Kubelet:** Agent on worker node. Talks to Docker/Containerd. 5. **Kube-Proxy:** Handles networking (IPTables).

Q48. Design a Log Aggregation System (ELK Stack)

[MEDIUM] | *Category: System Design*

Answer: We have 100 Microservices generating 1TB logs/day. We need to search them ("Find Error in Payment"). **The Elevator Pitch:** We cannot SSH into servers and `grep`. We need a centralized pipeline. **Stack:** 1. **Filebeat:** Agent on server. Tails log files. Pushes to Kafka. 2. **Kafka:** Buffers logs. 3. **Logstash/Fluentd:** Parser. Converts unstructured text to JSON. Geo-IP lookup. 4. **Elasticsearch:** Indexing. 5. **Kibana:** UI. **Architecture:** The pipeline decouples production (App) from analysis (Elastic). If Elastic is slow, Kafka buffers the logs so the App doesn't block.

Q49. Design a CI/CD Pipeline (Jenkins/GitLab)

[MEDIUM] | Category: System Design

Answer: We need to automate Build, Test, and Deploy. **The Elevator Pitch:** It's a **Directed Acyclic Graph (DAG)** of jobs. Commit -> Build (Docker) -> Unit Test -> Integration Test -> Push to Registry -> Deploy to Staging -> Manual Approval -> Deploy to Prod. **Components:** 1. **VCS Webhook:** GitHub triggers the pipeline. 2. **Orchestrator:** Jenkins Master/GitLab Runner. 3. **Worker Pool:** Docker containers that run the build. 4. **Artifact Store:** Registry (JFrog/ECR). **Isolation:** Every build runs in a fresh Docker container. No "It works on my machine" issues. State is discarded after build.

Q50. Design a Distributed Counter

[MEDIUM] | Category: Distributed Systems

Answer: We need to count likes/views for a Facebook post. 100k clicks/sec. **The Elevator Pitch:** We cannot `UPDATE table SET count = count + 1` in SQL. Row locking limits us to ~500/sec. **Solution 1:** Redis. Faster (100k/sec). But if Redis dies, we lose counts. **Solution 2: Sharded Counters.** We split the counter into N sub-counters. `Count = Counter_1 + Counter_2 + ... + Counter_N`. When writing, pick a random counter and increment. When reading, sum them up. **Architecture:** 1. **Ingestion:** API receives "Like". 2. **Sharding:** Hash(MessageID) -> Select Counter Shard. 3. **Persistence:** Write-Behind to SQL.

Q51. Design a Pastebin (Text Sharing Service)

[EASY] | Category: System Design

Answer: Pastebin allows users to upload text and get a unique URL to share it. The system is read-heavy (more people view pastes than create them). **Core Requirements:** 1. **Paste:** User uploads text -> System returns `url/xyz`. 2. **View:** User visits `url/xyz` -> System returns text. 3. **Expiration:** Pastes expire after X days. **High-Level Architecture:** 1. **Load Balancer:** Distributes traffic. 2. **App Server:** Handles the API (`createPaste`, `getPaste`). 3. **Database:** Stores metadata (Author, Expiration). 4. **Object Store (S3):** Stores the actual text content. Storing large text blobs in a SQL DB is inefficient; S3 is cheaper and scales better. 5. **Key Generator:** A service to pre-generate unique 6-char strings to avoid collision checks during write.

Q52. Design an Image Hosting Service (Imgur)

[EASY] | Category: System Design

Answer: This system hosts images uploaded by users. It requires high storage capacity and fast read speeds. ****Core Components:**** 1. ****Image Store:**** Object Storage (S3/GCS) is the only viable option for billions of files. 2. ****Metadata DB:**** Relational DB to track Owner, Upload Date, and Views. 3. ****CDN:**** Essential. Images must be cached at the edge to load fast. ****Workflow:**** * ****Upload:**** Client sends image -> Server -> S3. * ****Read:**** Client -> CDN -> (if miss) Server -> S3. ****Optimization:**** We create ****Thumbnails**** on upload. If a user uploads a 4K image, we shouldn't serve that 4K image to a mobile phone thumb scrolling the feed. We generate `small`, `medium`, and `large` versions immediately.

Q53. Design a Simple Logging Service

[EASY] | Category: System Design

Answer: We need a centralized place to store logs from 50 microservices for debugging. ****Requirements:**** 1. ****Ingestion:**** HTTP API to receive logs. 2. ****Storage:**** Append-only storage. 3. ****Search:**** Basic text search. ****Architecture:**** 1. ****Agent:**** A sidecar container (e.g., Fluentd) runs with the app, reads `stdout`, and sends to our service. 2. ****Queue:**** Kafka/RabbitMQ buffers the logs. 3. ****Indexer:**** Reads from Queue, writes to Elasticsearch. 4. ****UI:**** Kibana (or a custom React app) to search. ****Why the Queue?**** If the logging database goes down, the application shouldn't crash. The Queue acts as a buffer.

Q54. Design a Voting System (Polls)

[EASY] | Category: System Design

Answer: A system to create polls and vote. Strict requirement: ****One user, one vote.**** ****Core Components:**** 1. ****Poll Service:**** CRUD for polls. 2. ****Vote Service:**** Handles the casting logic. 3. ****Database:**** Needs consistency to count votes accurately. ****Architecture:**** * ****Write Path:**** User clicks vote -> API -> DB. * ****Read Path:**** The results page. This is read-heavy. We shouldn't `COUNT(\*)` every time. We cache the results. ****Consistency:**** We need transactional integrity. When User A votes, we must insert a record into `Votes` table AND increment the `Poll\_Counts` table.

Q55. Design a User Session System (Auth)

[EASY] | Category: System Design

Answer: How do we keep a user "logged in" across HTTP requests? **Approaches:** 1. **Stateful (Session DB):** Server creates a random `SessionID`, stores it in a DB/Redis map to the `UserID`. Sends `SessionID` as a Cookie. 2. **Stateless (JWT):** Server signs a JSON object (`{user_id: 123}`) with a secret key. Sends token to client. Client sends token back. Server verifies signature. **Recommendation:** For Easy HLD, **Stateful with Redis** is often simpler to reason about for revocation (logging out). **Architecture:** Client -> Load Balancer -> Auth Service -> Redis.

Q56. Design a 'Who is Online' Service

[EASY] | Category: System Design

Answer: We need to show a green dot next to friends who are active. **The Elevator Pitch:** This is a **Heartbeat System**. We cannot trust the client to say "I logged out" (browser crash, internet loss). Instead, the client must ping the server every X seconds: "I am still here". If we miss 3 pings, we mark them offline. **Architecture:** 1. **Heartbeat API:** Receives pings. 2. **Storage:** Redis (with TTL). 3. **Query:** "Get status of these 10 friends". **Why Redis?** SQL is too slow for updating `LastSeen` timestamp 1 million times a second. Redis `SETEX` (Set with Expiry) is atomic and fast.

Q57. Design a Simple Job Scheduler (Cron)

[EASY] | Category: System Design

Answer: We need to run tasks at specific times (e.g., "Send email at 9 AM"). **The Challenge:** If we have 3 servers, we don't want all 3 to send the email. We need **Mutual Exclusion**. **Architecture:** 1. **Job DB:** Stores tasks `(ID, ExecuteTime, Status)`. 2. **Poller Service:** Runs on multiple servers. Checks DB for due jobs. 3. **Locking:** Ensures only one server picks the job. **Simple Locking:** Use a database row lock. `UPDATE jobs SET status='RUNNING' WHERE id=1 AND status='PENDING'`. Only one update will succeed.

Q58. Design a Content Moderation System

[EASY] | Category: System Design

Answer: Users upload comments/images. We need to filter bad content before it goes public (or immediately after). **The Elevator Pitch:** This is a **Pipeline**. We shouldn't block the user's upload waiting for AI. Upload -> Queue -> Processors (AI/Human) -> Publish. **Architecture:** 1. **API:** Accepts content. Mark status as `PENDING_REVIEW`. 2. **Queue:** Kafka/SQS. 3. **Auto-Mod Worker:** Runs Regex/AI models (e.g., Google Vision API). 4. **Manual-Mod:** Dashboard for humans if AI is unsure. 5. **Action:** Update status to `APPROVED` or `REJECTED`. **User Experience:** Show the content to the *author* immediately (Optimistic UI), but hide it from *others* until approved.